

Infografía

Módulos 3 + 4 — Game loop, eventos, sprites

UPB · Gestión II/2025

Plan de la segunda mitad

1. **Recap rápido** — qué es el game loop y por qué `arcade.run()` no termina.
2. **Eventos** — teclado, mouse, (controller si hay).
3. **Salto a sprites** — de primitivas dibujadas a **imágenes**.
4. `hello_sprite.py` — la primera imagen en pantalla, con teclado.

*Acabamos de pasar la mañana **bajando** al pixel. Ahora **subimos**: dejamos de pintar pixel por pixel y empezamos a manipular objetos enteros (sprites).*

1. Game loop (recap)

El salto mental

Hasta el módulo 2 nuestros programas:

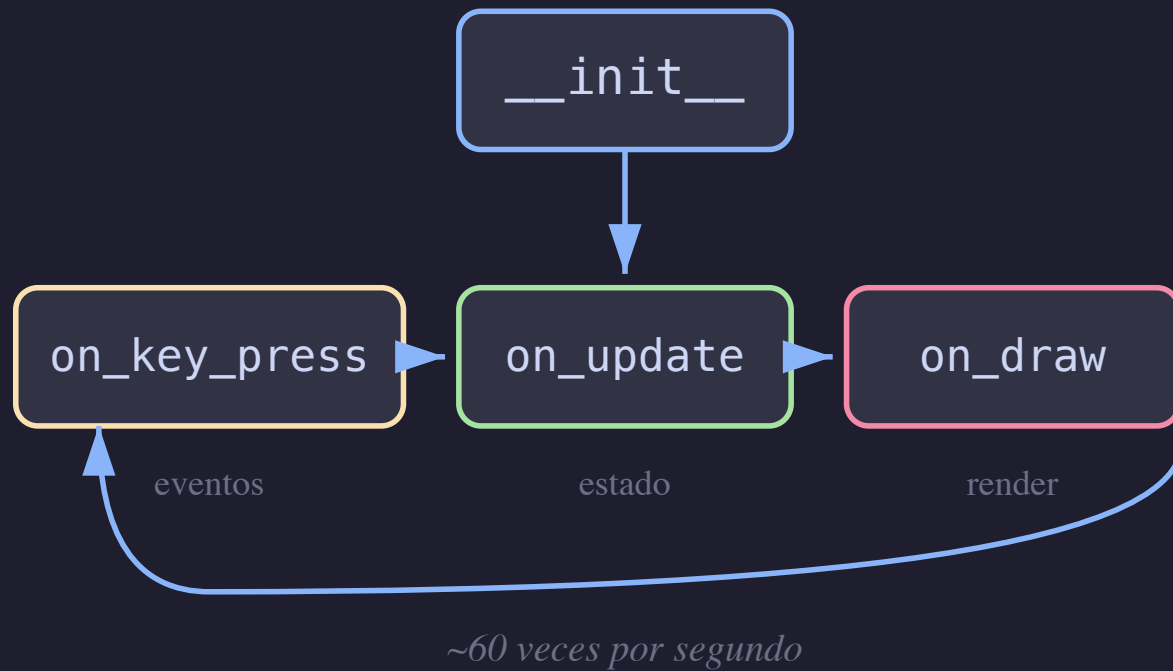
```
input -> calcular -> imprimir -> fin
```

Desde arcade:

```
init -> [ input -> update -> draw ] x60 por segundo -> ...
```

`arcade.run()` cede el control a un **bucle infinito** que llama a tus callbacks. El programa **ya no termina** al final del archivo.

El game loop



La estructura mínima

```
import arcade

class HelloView(arcade.View):
    def __init__(self):
        super().__init__()
        self.background_color = arcade.color.AMAZON

    def on_draw(self):
        self.clear()

window = arcade.Window(1280, 720, "hola arcade")
window.show_view(HelloView())
arcade.run()
```

Tres responsabilidades por View: `__init__` (estado), `on_event` (mutar estado), `on_draw` (renderizar estado).

`on_update` — el callback que cierra el loop

```
def on_update(self, delta_time: float):  
    self.ball.x += self.ball.vx * delta_time  
    self.ball.y += self.ball.vy * delta_time
```

- Arcade lo llama **automáticamente** ~60 veces por segundo.
- `delta_time` = segundos transcurridos desde el frame anterior.
- Multiplicar por `delta_time` hace el movimiento **independiente del framerate**.

Sin on_update: el estado solo cambia cuando el usuario hace algo.

Con on_update: el mundo tiene tiempo propio.

2. Eventos

teclado · mouse · controller

El patrón de eventos

```
def on_key_press(self, symbol, modifiers):  
    if symbol == arcade.key.UP:  
        self.char.y += self.speed  
  
def on_mouse_press(self, x, y, button, modifiers):  
    print(f"click en {x}, {y}")  
  
def on_mouse_drag(self, x, y, dx, dy, _buttons, _modifiers):  
    self.stroke_points.append((x, y))
```

evento → mutar estado → on_draw lo dibuja en el siguiente frame

*Los callbacks **nunca dibujan directo**. Solo cambian variables. La separación es el patrón clave del módulo.*

Eventos disponibles (los más usados)

Evento	Cuándo dispara
<code>on_key_press(symbol, mods)</code>	tecla bajada
<code>on_key_release(symbol, mods)</code>	tecla soltada
<code>on_mouse_press(x, y, button, mods)</code>	click
<code>on_mouse_release(x, y, button, mods)</code>	soltar click
<code>on_mouse_motion(x, y, dx, dy)</code>	mover mouse
<code>on_mouse_drag(x, y, dx, dy, buttons, mods)</code>	mover con click
<code>on_update(delta_time)</code>	cada frame (no es input, pero usa el mismo patrón)

Press vs. release: el patrón "dirección"

Para movimiento continuo conviene usar **ambos**:

```
def on_key_press(self, symbol, _mods):  
    if symbol == arcade.key.RIGHT:  
        self.player.change_x = MOVEMENT_SPEED  
  
def on_key_release(self, symbol, _mods):  
    if symbol == arcade.key.RIGHT:  
        self.player.change_x = 0
```

Y en `on_update`: `self.player.center_x += self.player.change_x`.

Si solo manejan `on_key_press`, el personaje se mueve un poco por cada keystroke (estilo "máquina de escribir"), no fluido.

Demo

```
cd 3._arcade  
uv run basic_events.py
```

Lo que hace:

- flechas arriba/abajo → mover círculo.
- click → empezar trazo nuevo.
- arrastrar → seguir el mouse con una polilínea verde.
- soltar mouse → teleportar el círculo a ese punto.

*El "teleport" al soltar es justo la motivación de `on_update`: ¿y si en vez de teletransportar quisiera **animar** el viaje? → necesitamos tiempo.*

3. Sprites

de píxeles dibujados a **imágenes**

¿Qué cambia al pasar a sprites?

Hasta ahora:

```
arcade.draw_circle_filled(self.x, self.y, r, color)
arcade.draw_line_strip(self.stroke_points, color, 3)
```

Dibujamos **figuras geométricas** con primitivas.

Con sprites:

```
self.sprite = arcade.Sprite("img/mario.png", scale=0.5)
arcade.draw_sprite(self.sprite)
```

Cargamos **una imagen** (PNG con transparencia) y la dibujamos. Misma posición, escala, rotación — pero el contenido es una textura.

Por qué importa

- Las imágenes traen **detalle visual gratis** (sombras, formas no-geométricas, animación).
- El motor las dibuja como **un cuadrado con textura** — internamente: dos triángulos que pasan por el rasterizador (que ahora entienden).
- Activan la pipeline real del GPU: textura, alpha blending, transform.

Sprite = el rectángulo texturizado que el GPU sabe dibujar más rápido que cualquier otra cosa

arcade.Sprite — atributos clave

```
self.sprite = arcade.Sprite("4._sprites/img/mario.png", scale=0.5)

self.sprite.center_x = 640          # posición
self.sprite.center_y = 360
self.sprite.scale     = 1.0         # tamaño relativo a la imagen original
self.sprite.angle     = 0           # rotación en grados (sentido antihorario)
```

Para dibujarlo en `on_draw`:

```
arcade.draw_sprite(self.sprite)
```

El path es **relativo al directorio desde donde corren python**. Por eso en este curso siempre corremos desde la carpeta del módulo: `cd 4._sprites && uv run hello_sprite.py`.

hello_sprite.py — la estructura

```
class SpriteView(arcade.View):  
    def __init__(self):  
        super().__init__()  
        self.background_color = arcade.color.BLACK  
        self.sprite = arcade.Sprite("4._sprites/img/mario.png", scale=0.5)  
        self.sprite.center_x = WIDTH // 2  
        self.sprite.center_y = HEIGHT // 2  
  
    def on_draw(self):  
        self.clear()  
        arcade.draw_sprite(self.sprite)
```

Mismo patrón de siempre: estado en `__init__`, render en `on_draw`.

hello_sprite.py — eventos

```
def on_key_press(self, symbol, modifiers):  
    if symbol == arcade.key.NUM_1:  
        self.sprite.scale = 1  
    elif symbol == arcade.key.NUM_2:  
        self.sprite.scale = 2  
    elif symbol == arcade.key.RIGHT:  
        self.sprite.center_x += 10  
    elif symbol == arcade.key.LEFT:  
        self.sprite.center_x -= 10  
    elif symbol == arcade.key.R:  
        self.sprite.angle += 5
```

Los mismos callbacks de antes — solo que ahora mutamos atributos de un Sprite, no de un CircleCharacter

Demo

```
cd 4._sprites  
uv run hello_sprite.py
```

Probar:

- **1 / 2** → cambiar escala (¿qué pasa con el centro del sprite?).
- **← / →** → mover horizontal.
- **R** → rotar 5° por keystroke.

Preguntas para clase:

- ¿Por qué con flechas se mueve "a saltos" en vez de fluido? → patrón press/release + `on_update`.
- ¿Qué sistema de coordenadas tiene el sprite? → centro, no esquina.
- ¿Qué pasa si la imagen no existe? → traceback amistoso de arcade.

El próximo paso (cliffhanger)

`hello_sprite.py` tiene **un solo sprite**. Para un juego real necesitamos:

- **Muchos sprites** organizados en `arcade.SpriteList` (dibujo en batch, rápido).
- **Velocidad** (`change_x`, `change_y`) y `on_update` para movimiento fluido.
- **Colisiones** entre sprites (jugador agarra moneda).
- **Spawn / destrucción** dinámica.

Ya está en el repo: `4._sprites/keyboard_movement.py` . Lo abrimos la próxima clase.

Resumen de la clase

Primera mitad (rasterización):

- Círculo (midpoint), Triángulo (scanline), Línea AA (Wu).
- Los tres algoritmos que el GPU corre por debajo.

Segunda mitad (arcade):

- Game loop: callbacks, no `print`.
- Eventos: mutar estado, dejar que `on_draw` lo refleje.
- Sprite: el rectángulo texturizado, la primitiva real del 2D moderno.

Bajamos al pixel y subimos al sprite — la misma pipeline, dos niveles distintos

¿Preguntas?

Próxima clase — `SpriteList`, colisiones, primer mini-juego

Tarea: jugar con `hello_sprite.py`. Agregar **una segunda imagen** (moneda) en otra posición.